Natural Language Processing

Hrachya Asatryan

What is NLP?

NLP is a subfield of **linguistics**, **computer science** and **artificial intelligence**, concerned with **interactions between computers and human language**.

What is NLP?

NLP is a subfield of **linguistics**, **computer science** and **artificial intelligence**, concerned with **interactions between computers and human language**.

Major subfields

- Speech recognition
- Natural language understanding (NLU) - we will mostly cover this topic
- Natural language generation (NLG) - we will touch on some aspects of this

What is NLP?

NLP is a subfield of **linguistics**, **computer science** and **artificial intelligence**, concerned with **interactions between computers and human language**.

Major subfields

- Speech recognition
- Natural language understanding (NLU) - we will mostly cover this topic
- Natural language generation (NLG) - we will touch on some aspects of this

Many applications are a mixture of the above:

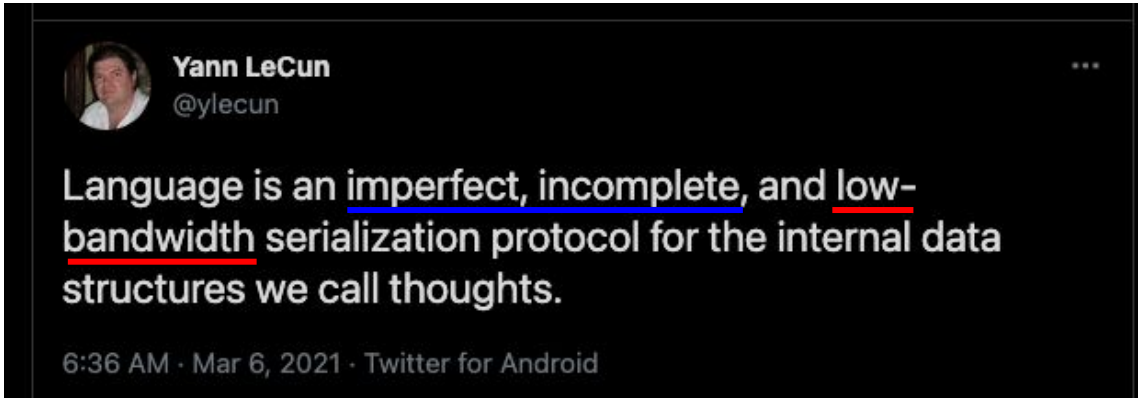
e.g. chatbots (NLU + NLG), Google Assistant (speech recognition + NLU)

What is NLP?

- Most of the knowledge we have has been communicated to us through human language
  - We also learn many things by seeing (computer vision) and interacting with the world around us (reinforcement learning), however...
- Language took us from basic tools to spaceships and nanotechnology
  - Vision: ~500,000,000 years
  - Spoken language: ~100,000 years
  - Written language ~5,200 years (in the area of present day Iraq)

What is language?

- "...a large network computer where the communication protocol is the human language.", "...a compression protocol, assuming vast amount of some pre-existing knowledge." - paraphrasing C. Manning



History of NLP

Symbolic NLP (1950s~1990s)

- Rule-based, e.g. a large dictionary and word matching, pre-defined question/answer pairs
- Rule-based dependency parsing of sentences
- Conceptual ontologies (e.g. knowledge graphs are build on top of ontologies) [Wikipedia]

Statistical NLP (1990s~2010s)

- Mostly denotational semantics| signifier (symbol) -> signified (idea or a thing)
- Sparse, categorical representations, one-hot-vectors + statistical methods

Modern NLP (since 2012/2013)

- Distributional semantics | "You shall know a word by the company it keeps" - J.R. Firth (1957)
- Neural NLP

Applications of NLP

- Word meaning (e.g. synonyms, related words) - important for other applications!
- Document classification (topic classification, sentiment analysis)
- Sequence tagging (part of speech tagging, named entity recognition)
- Text mining (e.g. relationship extraction)
- Machine translation
- Question answering
- Text summarization
- **Chatbots**

Word meaning

Definition: **meaning** (Webster dictionary)

- the idea that is represented by a word, phrase, etc.
- the idea that a person wants to express by using words, signs, etc.
- the idea that is expressed in a work of writing, art, etc.

Denotational semantics

- signifier (symbol) -> signified (idea or a thing)

e.g. "chair" (symbol) -> (thing)




WordNet

```
import nltk
nltk.download('wordnet')

from nltk.corpus import wordnet as wn
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
[2] parts_of_speech = {
        'n': 'noun', 'v': 'verb', 's': 'adj (s)', 'a': 'adj', 'r': 'adv'}

    def print_synonyms(w):
      for synset in wn.synsets(w):
        print(f'{parts_of_speech[synset.pos()]}:'
              f' {", ".join([l.name() for l in synset.lemmas()])}')
```

```
[3] print_synonyms('good')
```

```
noun: good
noun: good, goodness
noun: good, goodness
noun: commodity, trade_good, good
adj: good
adj (s): full, good
adj: good
adj (s): estimable, good, honorable, respectable
adj (s): beneficial, good
adj (s): good
adj (s): good, just, upright
adj (s): adept, expert, good, practiced, proficient, skillful, skilfu
adj (s): good
adj (s): dear, good, near
adj (s): dependable, good, safe, secure
adj (s): good, right, ripe
adj (s): good, well
adj (s): effective, good, in_effect, in_force
```

hypernym: a word with a broad meaning that more specific words fall under.

```
def get_hypernyms(w):
  word = wn.synset(f'{w}.n.01')
  hyper = lambda s: s.hypernyms()
  return list(word.closure(hyper))
```

```
[6] get_hypernyms('panda')
```

```
[Synset('procyonid.n.01'),
 Synset('carnivore.n.01'),
 Synset('placental.n.01'),
 Synset('mammal.n.01'),
 Synset('vertebrate.n.01'),
 Synset('chordate.n.01'),
 Synset('animal.n.01'),
 Synset('organism.n.01'),
 Synset('living_thing.n.01'),
 Synset('whole.n.02'),
 Synset('object.n.01'),
 Synset('physical_entity.n.01'),
 Synset('entity.n.01')]
```

```
[7] get_hypernyms('red')
```

```
[Synset('chromatic_color.n.01'),
 Synset('color.n.01'),
 Synset('visual_property.n.01'),
 Synset('property.n.02'),
 Synset('attribute.n.02'),
 Synset('abstraction.n.06'),
 Synset('entity.n.01')]
```

WordNet

Drawbacks

- Impossible to maintain and keep up-to-date
  - new words, new meanings of words
- Requires a lot of human labor
- Subjective
- Can't compute accurate word similarity

Denotational semantics

- Words are discrete symbols, so they are often encoded as **one-hot-vectors**

```
cat = [0 0 0 1 0 0 0 0]
dog = [0 0 0 0 0 1 0 0]
```

Issues:

- Languages have many words! (a very sparse representation)
- Similarity between words is not encoded (e.g. cat and dog are very similar compare to e.g. cat and spaceship)

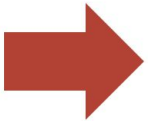
Solution: encode similarity in vectors themselves

Distributed Representations

- Map each word to a vector in space

Dimensions correspond to some attributes (features) of words

Vocabulary:
Man, woman, boy, girl, prince, princess, queen, king, monarch



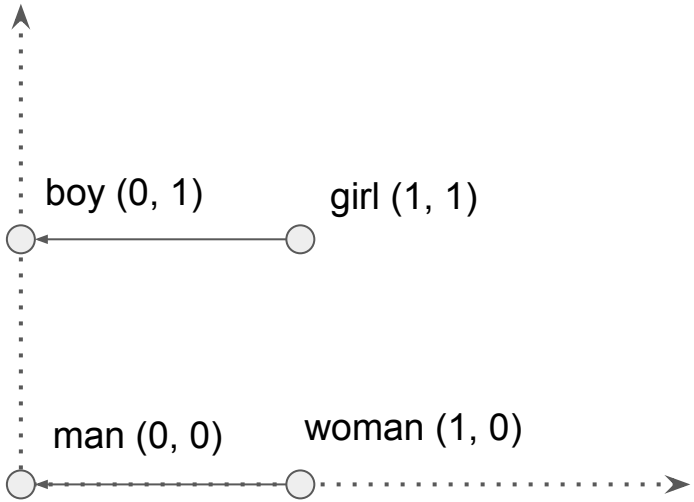
	Femininity	Youth	Royalty
Man	0	0	0
Woman	1	0	0
Boy	0	1	0
Girl	1	1	0
Prince	0	1	1
Princess	1	1	1
Queen	1	0	1
King	0	0	1
Monarch	0.5	0.5	1

Each word gets a 1x3 vector

Similar words… similar vectors

Distributed Representations

	Femininity	Youth	Royalty
Man	0	0	0
Woman	1	0	0
Boy	0	1	0
Girl	1	1	0
Prince	0	1	1
Princess	1	1	1
Queen	1	0	1
King	0	0	1
Monarch	0.5	0.5	1



girl - boy = woman - man

Distributed Representations

	Femininity	Youth	Royalty
Man	0	0	0
Woman	1	0	0
Boy	0	1	0
Girl	1	1	0
Prince	0	1	1
Princess	1	1	1
Queen	1	0	1
King	0	0	1
Monarch	0.5	0.5	1

Other linear relationships

- queen - king = woman - man = girl - boy
- boy + woman = girl
- (prince + princess + queen + king) / 4 = monarch
- ...

How can we learn the distributed representation?

Distributional Semantics

"You shall know a word by the company it keeps" - J.R. Firth (1957)

- A word's meaning is given by the words that frequency appear close-by
- One of the most successful ideas of modern NLP

… միլիարդ աշակերտ այսօր դպրոց չի հաճախում, քանի որ …

… տարրական դպրոց ընդունված մեկ աշակերտի ուսման …

Նման կերպ հանրակրթական դպրոց են ընդունվում 6 տարեկանում և …

these context words represent the meaning of դպրոց

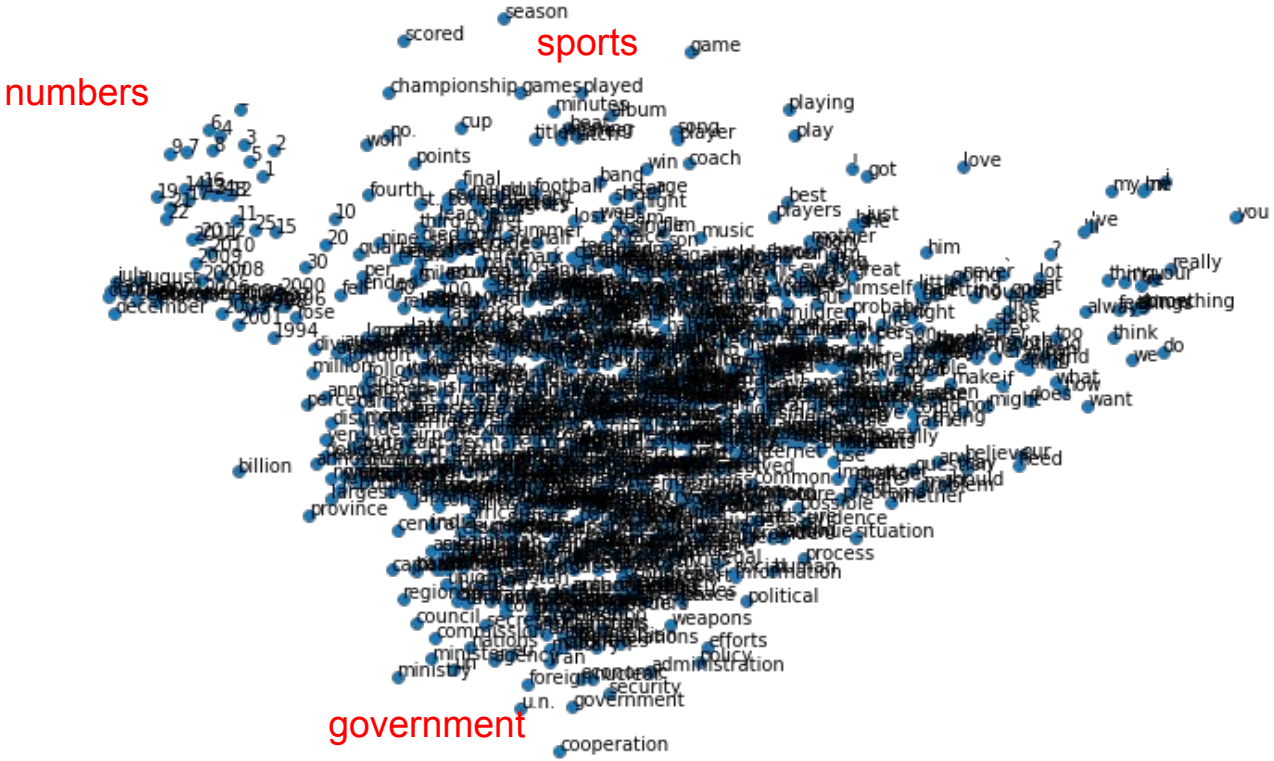
Word Embeddings

- Often also called **word embeddings**

$$\text{ηψηng} = \begin{bmatrix} 0.256 \\ 0.854 \\ -0.234 \\ -0.232 \\ 0.123 \\ 0.234 \\ -0.542 \\ 0.171 \end{bmatrix}$$

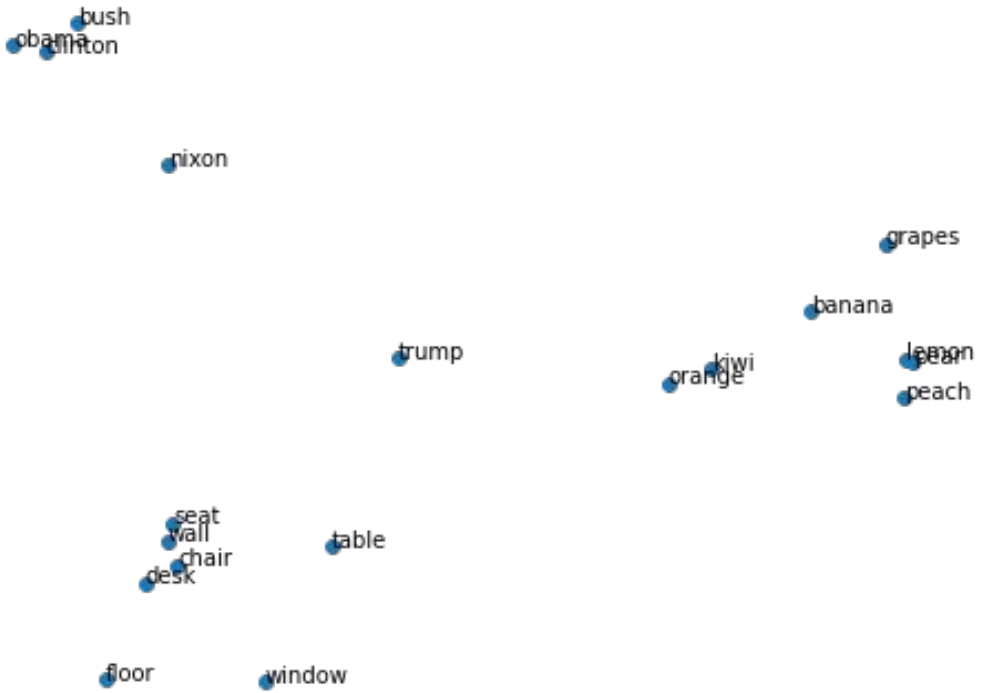
- The dimensionality is usually much smaller than the size of the vocabulary (50~1000)
- Similar words will have similar embeddings (small distance in vector space)

Word Embeddings

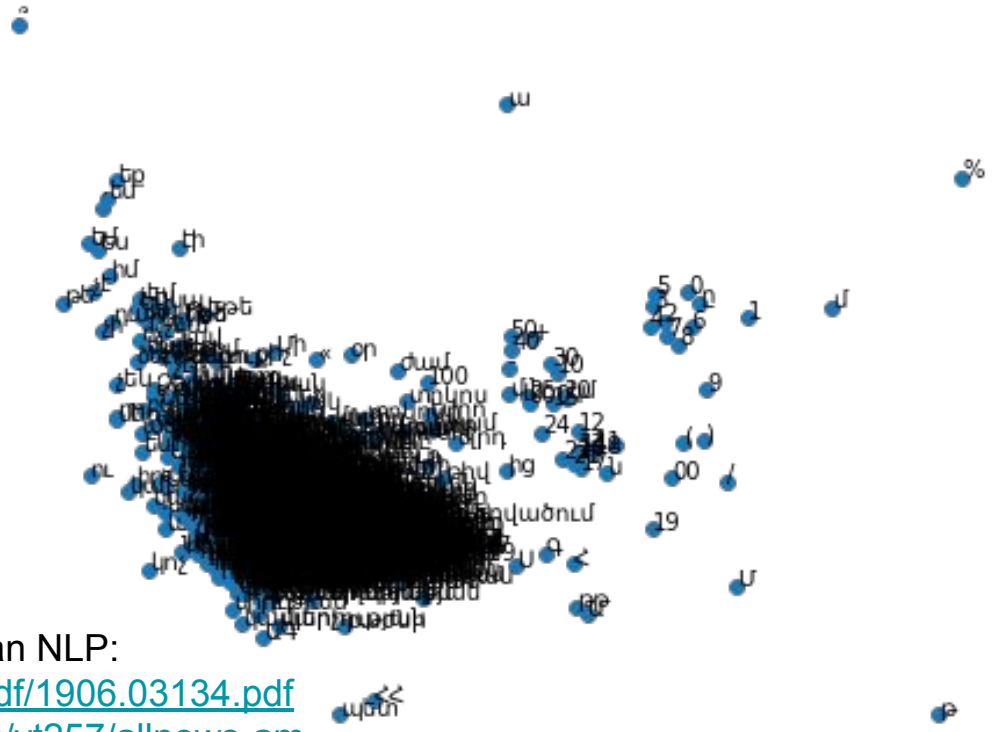


Word Embeddings

We can also pick subsets of words



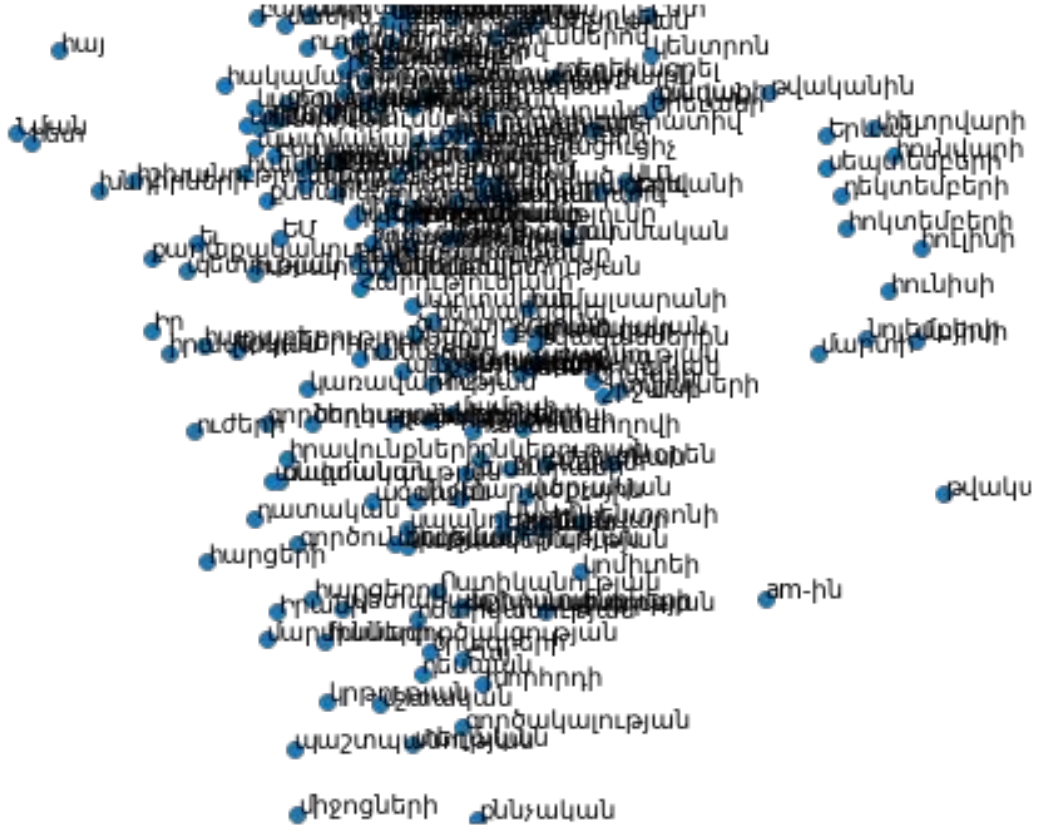
Armenian Word Embeddings



Useful links for Armenian NLP:

- https://arxiv.org/pdf/1906.03134.pdf
- https://github.com/vt257/allnews-am
- https://github.com/YerevaNN/word2vec-armenian-wiki

Armenian News Word Embeddings

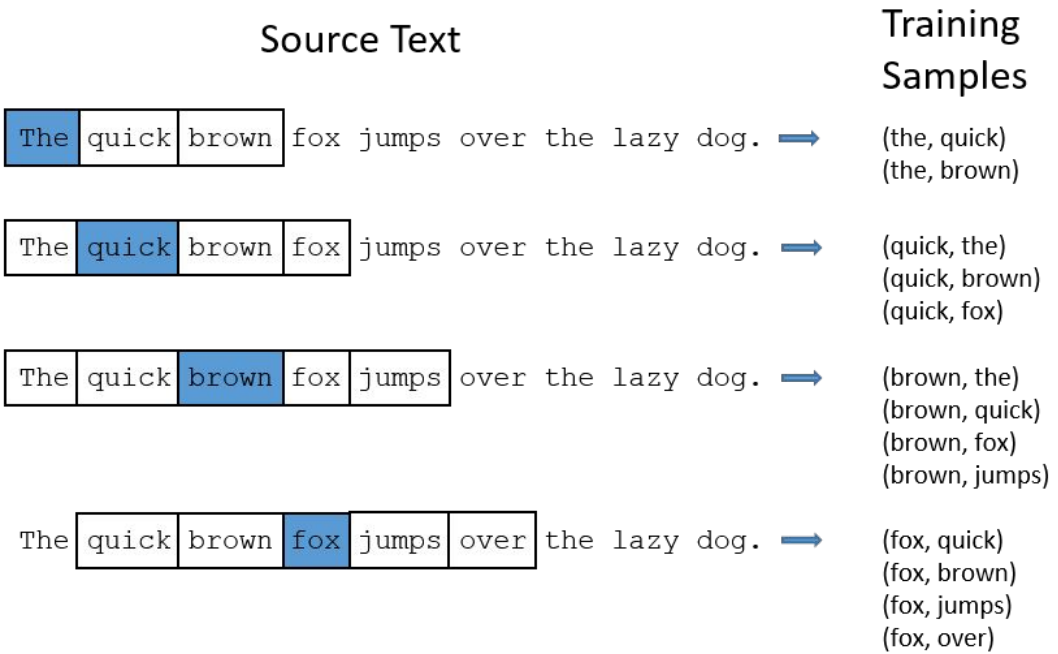


Learning word embeddings with Word2vec skip-gram

Word2Vec

1. Take a large text corpus
2. Assume a fixed vocabulary, with 1 vector per word

Train a 1-hidden-layer network to predict context words* given a target word for every position in the text corpus.

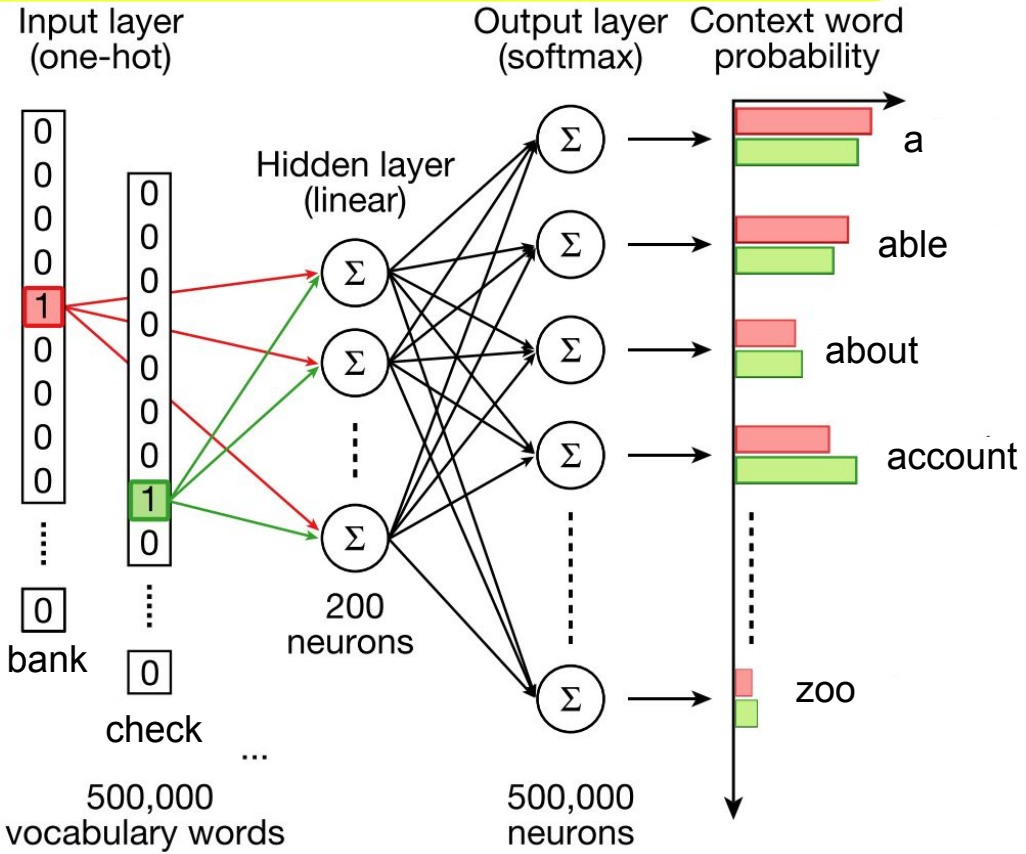


- Mikolov et al. arXiv:1301.3781 (2013)
- http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/

*context words are words within a fixed-size window, a hyperparameter usually chosen to be 2 - 10

Word2Vec

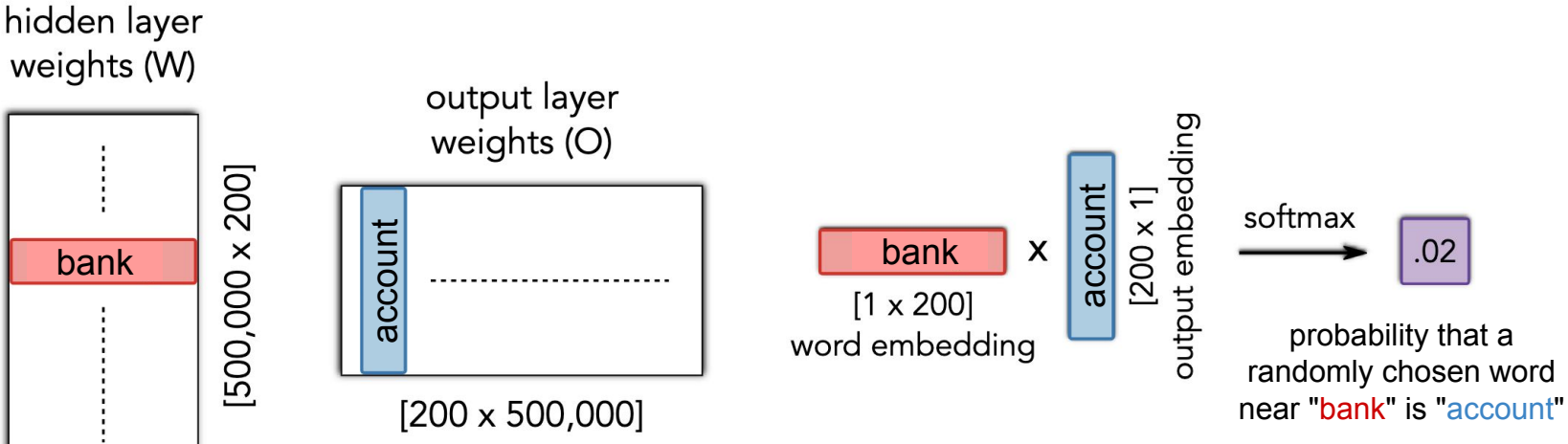
- words with similar meanings → mostly the same context words
- → similar hidden layer weights for the input neurons corresponding to the one-hot positions of similar words in the vocabulary
- The hidden layer weights are the word embeddings



*no activation, no bias neurons

Word2Vec

2 sets of embeddings (200,000,000 total parameters)*



word vectors / embeddings / representations

"output embeddings" are often discarded, or sometimes averaged with word embeddings

*possible to do with single set, but the derivatives are not as nice as in this case

Similarity

Word similarity is usually defined as the dot product of the word vectors (projection of one on the other)

Often embeddings is normalized beforehand, so the similarity is only determined by the angle between 2 vectors

- cosine similarity

```
armenian_embeddings.wv.most_similar(positive=['դպրոց'])
```

```
[('դպրոցը', 0.9347216486930847),
 ('դպրոցում', 0.9304841756820679),
 ('դպրոցս', 0.927428662776947),
 ('Դպրոց', 0.9206417798995972),
 ('դպրոցը', 0.9072887301445007),
 ('գրադպրոց', 0.9066643118858337),
 ('դպրոցիս', 0.9043025970458984),
 ('դպրոցն', 0.9037590026855469),
 ('սպորտդպրոց', 0.8666008114814758),
 ('դպրոցիդ', 0.8576775193214417)]
```

```
[177] armenian_embeddings.wv.most_similar(positive=['կարմիր'])
```

```
[('Սև-կարմիր', 0.8918075561523438),
 ('կապույտ', 0.8345009088516235),
 ('արնակարմիր', 0.8093111515045166),
 ('դեղին', 0.8082975149154663),
 ('կարմիրի', 0.8051129579544067),
 ('նարնջակարմիր', 0.800564706325531),
 ('կապտակարմիր', 0.7926867604255676),
 ('շիկակարմիր', 0.7907490134239197),
 ('կարմրակապույտ', 0.7717244029045105),
 ('կարմիրն', 0.7715886235237122)]
```

Dissimilarity

Antonyms are one of the limitations of word2vec or similar context-based embedding approaches

```
armenian_embeddings.wv.most_similar(negative=['կարմիր'])
```

```
[('Մէ', 0.5067793130874634),
 ('էի', 0.49410906434059143),
 ('աշխատունակ', 0.4869403839111328),
 ('Յըլ', 0.4710981845855713),
 ('Ասատր', 0.45935001969337463),
 ('Շահսենեմ', 0.45517224073410034),
 ('104-ը', 0.4510742425918579),
 ('64', 0.44867807626724243),
 ('Էթեմ', 0.4444555640220642),
 ('onto', 0.4409807026386261)]
```

```
[179] armenian_embeddings.wv.most_similar(negative=['մեծ'])
```

```
[('02', 0.5880160927772522),
 ('05', 0.5584599375724792),
 ('07', 0.5572476387023926),
 ('Էլմ', 0.5519378781318665),
 ('01', 0.5506056547164917),
 ('Ազաբ', 0.5423334240913391),
 ('04', 0.5396718382835388),
 ('զեկուցագիր', 0.5393275022506714),
 ('08', 0.5384430289268494),
 ('06', 0.5373049378395081)]
```

Run ce

Analogies

Recall that **directions in space can have a meaning**, e.g.

king - queen = man - woman

The question here is

word1 is to *word2* is as *word3* is to ?

Mathematically

word2 - word1 + word3 = ?

```
analogy('Երևան', 'Հայաստան', 'Մոսկվա')
```

?

```
[188] analogy('Պուտին', 'Ռուսաստան', 'Փաշինյան')
```

?

```
[191] analogy('Հայաստան', 'հայ', 'ԱՄՆ')
```

?

```
[200] analogy('մեծ', 'փոքր', 'սև')
```

?

```
[221] analogy('ուսուցիչ', 'դպրոց', 'դասախոս')
```

?

```
[214] analogy('Գերմանիա', 'գարեջուր', 'Ֆրանսիա')
```

?

Analogies

Recall that **directions in space can have a meaning**, e.g.

king - queen = man - woman

The question here is

word1 is to *word2* is as *word3* is to ?

Mathematically

word2 - *word1* + *word3* = ?

```
analogy('Երևան', 'Հայաստան', 'Մոսկվա')
'Ռուսաստան'
```

```
[188] analogy('Պուտին', 'Ռուսաստան', 'Փաշինյան')
```

?

```
[191] analogy('Հայաստան', 'հայ', 'ԱՄՆ')
```

?

```
[200] analogy('մեծ', 'փոքր', 'սև')
```

?

```
[221] analogy('ուսուցիչ', 'դպրոց', 'դասախոս')
```

?

```
[214] analogy('Գերմանիա', 'գարեջուր', 'Ֆրանսիա')
```

?

Analogies

Recall that **directions in space can have a meaning**, e.g.

king - queen = man - woman

The question here is

word1 is to *word2* is as *word3* is to ?

Mathematically

word2 - *word1* + *word3* = ?

```
analogy('Երևան', 'Հայաստան', 'Մոսկվա')
'Ռուսաստան'
```

```
[188] analogy('Պուտին', 'Ռուսաստան', 'Փաշինյան')
'Հայաստան'
```

```
[191] analogy('Հայաստան', 'հայ', 'ԱՄՆ')
```

?

```
[200] analogy('մեծ', 'փոքր', 'սև')
```

?

```
[221] analogy('ուսուցիչ', 'դպրոց', 'դասախոս')
```

?

```
[214] analogy('Գերմանիա', 'գարեջուր', 'Ֆրանսիա')
```

?

Analogies

Recall that **directions in space can have a meaning**, e.g.

king - queen = man - woman

The question here is

word1 is to *word2* is as *word3* is to ?

Mathematically

word2 - *word1* + *word3* = ?

```
analogy('Երևան', 'Հայաստան', 'Մոսկվա')
'Ռուսաստան'
```

```
[188] analogy('Պուտին', 'Ռուսաստան', 'Փաշինյան')
'Հայաստան'
```

```
[191] analogy('Հայաստան', 'հայ', 'ԱՄՆ')
'ամերիկացի'
```

```
[200] analogy('մեծ', 'փոքր', 'սև')
```

?

```
[221] analogy('ուսուցիչ', 'դպրոց', 'դասախոս')
```

?

```
[214] analogy('Գերմանիա', 'գարեջուր', 'Ֆրանսիա')
```

?

Analogies

Recall that **directions in space can have a meaning**, e.g.

king - queen = man - woman

The question here is

word1 is to *word2* is as *word3* is to ?

Mathematically

word2 - *word1* + *word3* = ?

```
analogy('Երևան', 'Հայաստան', 'Մոսկվա')
'Ռուսաստան'
```

```
[188] analogy('Պուտին', 'Ռուսաստան', 'Փաշինյան')
'Հայաստան'
```

```
[191] analogy('Հայաստան', 'հայ', 'ԱՄՆ')
'ամերիկացի'
```

```
[200] analogy('մեծ', 'փոքր', 'սև')
'սպիտակ'
```

```
[221] analogy('ուսուցիչ', 'դպրոց', 'դասախոս')
```

?

```
[214] analogy('Գերմանիա', 'գարեջուր', 'Ֆրանսիա')
```

?

Analogies

Recall that **directions in space can have a meaning**, e.g.

king - queen = man - woman

The question here is

word1 is to *word2* is as *word3* is to ?

Mathematically

word2 - *word1* + *word3* = ?

```
analogy('Երևան', 'Հայաստան', 'Մոսկվա')
'Ռուսաստան'
```

```
[188] analogy('Պուտին', 'Ռուսաստան', 'Փաշինյան')
'Հայաստան'
```

```
[191] analogy('Հայաստան', 'հայ', 'ԱՄՆ')
'ամերիկացի'
```

```
[200] analogy('մեծ', 'փոքր', 'սև')
'սպիտակ'
```

```
[221] analogy('ուսուցիչ', 'դպրոց', 'դասախոս')
'պետհամալսարան'
```

```
[214] analogy('Գերմանիա', 'գարեջուր', 'Ֆրանսիա')
```

?

Analogies

Recall that **directions in space can have a meaning**, e.g.

king - queen = man - woman

The question here is

word1 is to *word2* is as *word3* is to ?

Mathematically

word2 - word1 + word3 = ?

```
analogy('Երևան', 'Հայաստան', 'Մոսկվա')
'Ռուսաստան'
```

```
[188] analogy('Պուտին', 'Ռուսաստան', 'Փաշինյան')
'Հայաստան'
```

```
[191] analogy('Հայաստան', 'հայ', 'ԱՄՆ')
'ամերիկացի'
```

```
[200] analogy('մեծ', 'փոքր', 'սև')
'սպիտակ'
```

```
[221] analogy('ուսուցիչ', 'դպրոց', 'դասախոս')
'պետհամալսարան'
```

```
[214] analogy('Գերմանիա', 'գարեջուր', 'Ֆրանսիա')
'շամպայն'
```

Bag of Tricks

- Some words are very common and can make the results worse for other words (e.g. 'the', 'a', 'am', …)
- Many training parameters - slow training for a seemingly simple model
- The distance of the word within a window doesn't matter
- Multi-word phrases with specific meaning, e.g. 'The New York Times'

Bag of Tricks

- When the training examples are generated, discard words with a probability that depends on their frequency in the text corpus
- For the Word2vec paper

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

for f(w) >= t, otherwise 0. t is a fixed threshold.

Bag of Tricks

- When the training examples are generated, discard words with a probability that depends on their frequency in the text corpus
- For the Word2vec paper

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

for f(w) >= t, otherwise 0. t is a fixed threshold.

… աշակերտ այսօր դպրոց չի հաճախում քանի ...

e.g. *t* = 1e-5 in the original paper (1 in 100,000 words)
assume:

```
f(չի) = 1e-3 (1 in 1,000 words is չի)-> P(չի) = 0.9 (ignored 90% of time)
f(աշակերտ) = 1e-4                    -> P(աշակերտ) = 0.69 (ignored 69% of time)
f(այսօր) = 1e-5                       -> P(այսօր) = 0 (always used)
f(հաճախում) = 1e-7                      -> P(հաճախում) = 0 (always used)
```

Bag of Tricks

- When the training examples are generated, discard words with a probability that depends on their frequency in the text corpus
- For the Word2vec paper

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

for f(w) >= t, otherwise 0. t is a fixed threshold.

… աշակերտ այսօր դպրոց ~~չի~~ հաճախում քանի ...

the examples could be:
(դպրոց, աշակերտ)
(դպրոց, այսօր)
(դպրոց, հաճախում)
(դպրոց, քանի)

e.g. *t* = 1e-5 in the original paper (1 in 100,000 words)
assume:

```
f(չի) = 1e-3 (1 in 1,000 words is չի)-> P(չի) = 0.9 (ignored 90% of time)
f(աշակերտ) = 1e-4                    -> P(աշակերտ) = 0.69 (ignored 69% of time)
f(այսօր) = 1e-5                       -> P(այսօր) = 0 (always used)
f(հաճախում) = 1e-7                      -> P(հաճախում) = 0 (always used)
```

Bag of Tricks

- Use a dynamic window size
- E.g. if the window size hyperparameter is 5
  - Assume a max_window_size = 5
  - Uniformly sample between 1, 2, 3, 4 and 5 window size.
- Close words will have higher influence
- e.g. for a window size of 2

… աշակերտ այսօր դպրոց չի հաճախում քանի ...

50% probability
(դպրոց, աշակերտ)
(դպրոց, այսօր)
(դպրոց, չի)
(դպրոց, հաճախում)

50% probability
(դպրոց, այսօր)
(դպրոց, չի)

Bag of Tricks

- Find words that appear frequently together, and infrequently on their own
- e.g. "New York Times" would become a phrase, but "this is" would not

$$\text{score}(w_i, w_j) = \frac{\text{count}(w_i w_j) - \delta}{\text{count}(w_i) \times \text{count}(w_j)}.$$

- **δ** is a "discounting threshold" to avoid phrases with very infrequent words
- bigrams (pairs) with a score greater than some threshold will be included as individual words
- repeat the process 2 - 4 times to get phrases with more than 2 words

Issues

- Words with multiple distinct meanings / senses
  - For most of the applications, this is not an issue. Different dimensions of the vector space can capture different meanings [e.g. Arora et al. (2018) separate word senses post-training]
  - If you really wanted to, you could cluster contexts of the same word, and identify the senses before the training. You will end up with something like bush1, bush2, … [Huang et al. ACL 2012)
- Out-of-vocabulary words
  - FastText - Word2vec skip gram using character n-grams in addition to words.
  - When there is an out-of-vocabulary word, just sum up the embeddings for character n-grams
  - e.g. revolutionary = revolution + ary

Co-occurrence Matrices

- Compute word-word co-occurrence matrix
  - Sparse
  - Large memory requirements
- Perform dimensionality reduction, e.g. SVD (latent semantic analysis, LSA)
- Relevant papers: LSA (1988), HAL (1996, introduces ramped windows)
- E.g. for 2 sentences: "I like NLP" and "I like DL"

/	I	like	NLP	DL
I	0	2	1	1
like	2	0	1	1
NLP	1	1	0	0
DL	1	1	0	0

NLP and DL get the same vectors!

GloVe: Global Vectors

- This paper followed Word2vec paper, trying to combine the count-based approaches of LSA/HAL with the learning approach of Word2vec
- Explicitly optimizes for analogies (linear operations in vector space to capture relationships between words)
- Paper link

GloVe: Global Vectors

Crucial insight: Ratios of co-occurrence probabilities can encode meaning components

	x = solid	x = gas	x = water	x = fashion
$P(x\|\text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(x\|\text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$\frac{P(x\|\text{ice})}{P(x\|\text{steam})}$	8.9	8.5×10^{-2}	1.36	0.96

GloVe: Global Vectors

Q: How can we capture ratios of co-occurrence probabilities as linear meaning components in a word vector space?

A: Log-bilinear model: $w_i \cdot w_j = \log P(i|j)$

with vector differences $w_x \cdot (w_a - w_b) = \log \frac{P(x|a)}{P(x|b)}$

GloVe: Global Vectors

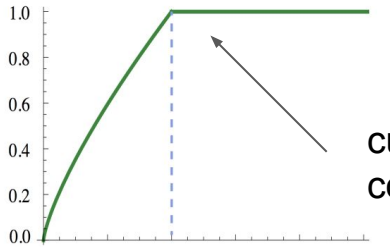
$$w_i \cdot w_j = \log P(i|j)$$

$$J = \sum_{i,j=1}^{V} f\left(X_{ij}\right)\left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2$$

bias terms, solving common / uncommon word issues

- Fast training
- Scalable to huge corpora
- Good performance even with small corpus and small vectors

$f \sim$



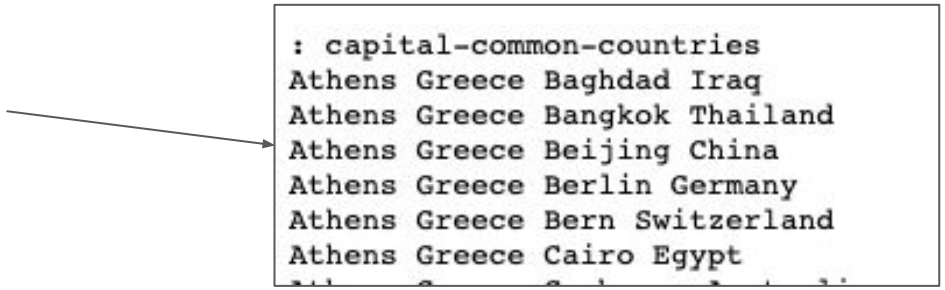
cutoff to threshold common pairs

Word2Vec vs GloVe

- GloVe explicitly optimizes for word analogies
- GloVe uses more memory (need to compute the matrix), but is often faster to train (easy to parallelize).
- Word2vec performs online training (low memory requirement), and it is easy to take a pre-trained model and continue training (no need for the original corpus).
- Word2vec is easier to understand*
- Both: issue with antonyms and out-of-vocab words.

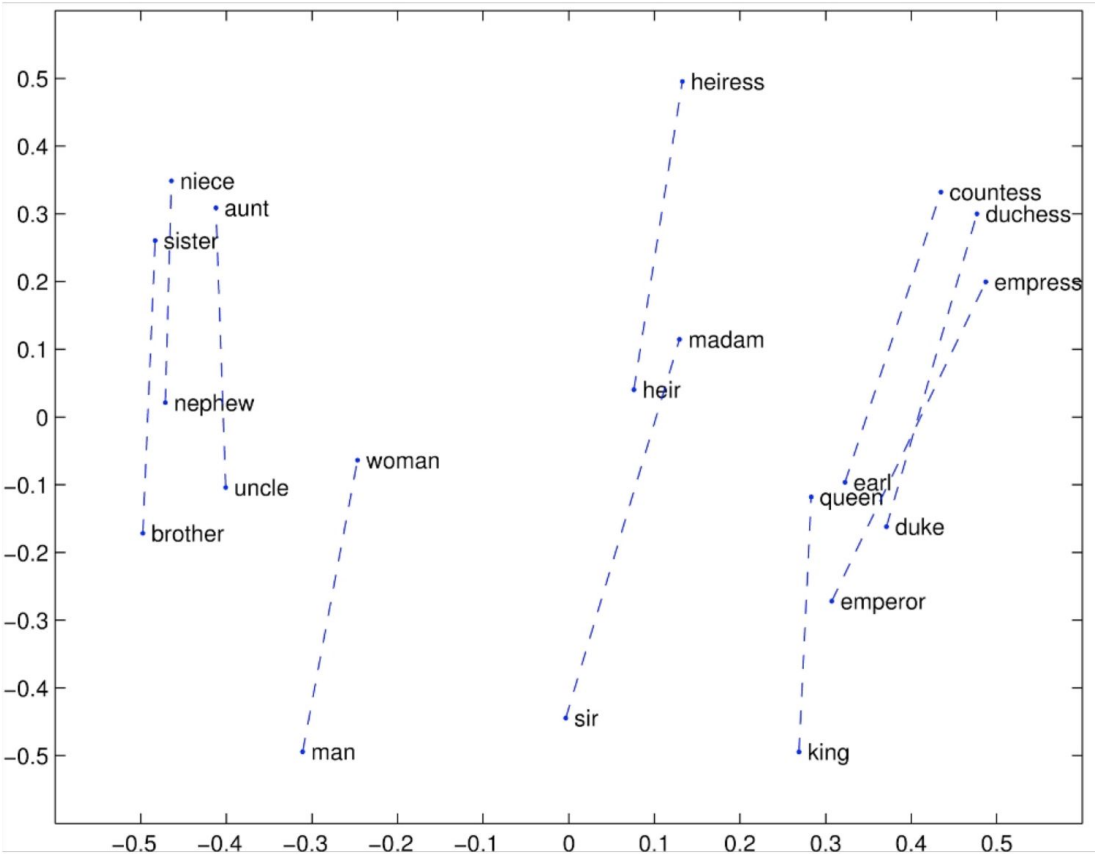
Evaluation

- Intrinsic evaluations
  - Analogy pairs (top 1, top 5, …)
  - Word similarity

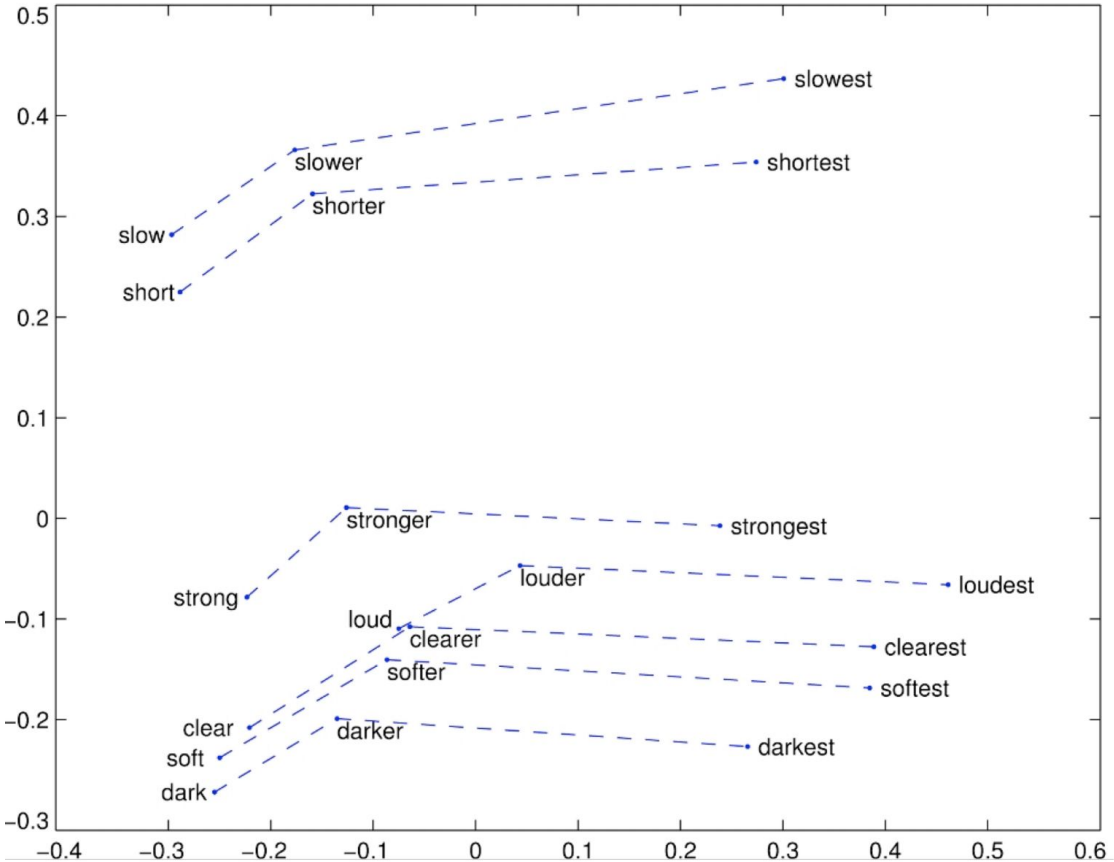


- Extrinsic evaluation on real tasks
  - Document classification
  - Named entity recognition

Analogies


niece
aunt
sister
nephew
uncle
brother
woman
man
heiress
madam
heir
sir
countess
duchess
empress
earl
queen
duke
emperor
king
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5

Analogies


slowest
slower
shortest
shorter
slow
short
stronger
strongest
louder
loudest
strong
loud
clearer
clearest
softer
softest
clear
darker
darkest
soft
dark
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5
0.6

Analogies

- From GloVe paper

Model	Dim.	Size	Sem.	Syn.	Tot.
ivLBL	100	1.5B	55.9	50.1	53.2
HPCA	100	1.6B	4.2	16.4	10.8
GloVe	100	1.6B	67.5	54.3	60.3
SG	300	1B	61	61	61
CBOW	300	1.6B	16.1	52.6	36.1
vLBL	300	1.5B	54.2	64.8	60.0
ivLBL	300	1.5B	65.2	63.0	64.0
GloVe	300	1.6B	80.8	61.5	70.3
SVD	300	6B	6.3	8.1	7.3
SVD-S	300	6B	36.7	46.6	42.1
SVD-L	300	6B	56.6	63.0	60.1
CBOW†	300	6B	63.6	67.4	65.7
SG†	300	6B	73.0	66.0	69.1
GloVe	300	6B	77.4	67.0	71.7
CBOW	1000	6B	57.3	68.9	63.7
SG	1000	6B	66.1	65.1	65.6
SVD-L	300	42B	38.4	58.2	49.2
GloVe	300	42B	**81.9**	**69.3**	**75.0**

naive SVD is not that great for this but if you tweak it, things get better!

Word2vec skip-gram (SG) also does OK in certain settings

GloVe is the best - this is part of the optimization objective!

Analogies

- From GloVe paper

Model	Dim.	Size	Sem.	Syn.	Tot.
ivLBL	100	1.5B	55.9	50.1	53.2
HPCA	100	1.6B	4.2	16.4	10.8
GloVe	100	1.6B	67.5	54.3	60.3
SG	300	1B	61	61	61
CBOW	300	1.6B	16.1	52.6	36.1
vLBL	300	1.5B	54.2	64.8	60.0
ivLBL	300	1.5B	65.2	63.0	64.0
GloVe	300	1.6B	80.8	61.5	70.3
SVD	300	6B	6.3	8.1	7.3
SVD-S	300	6B	36.7	46.6	42.1
SVD-L	300	6B	56.6	63.0	60.1
CBOW†	300	6B	63.6	67.4	65.7
SG†	300	6B	73.0	66.0	69.1
GloVe	300	6B	77.4	67.0	71.7
CBOW	1000	6B	57.3	68.9	63.7
SG	1000	6B	66.1	65.1	65.6
SVD-L	300	42B	38.4	58.2	49.2
GloVe	300	42B	**81.9**	**69.3**	**75.0**

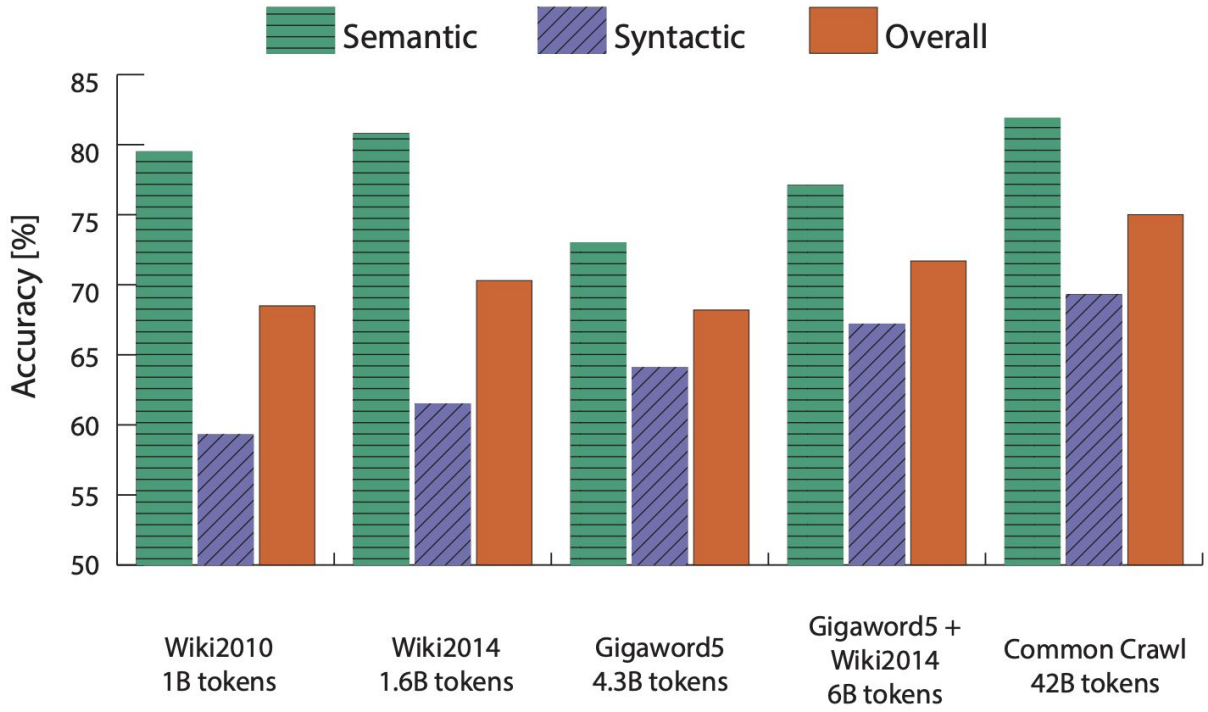
naive SVD is not that great for this but if you tweak it, things get better!

Word2vec skip-gram (SG) also does OK in certain settings

Follow up research showed that with correct hyperparameter tuning, GloVe and Word2vec are pretty much the same

GloVe is the best - this is part of the optimization objective!

More (good) data (usually) helps



Evaluation

- Intrinsic evaluations
  - Analogy pairs (top 1, top 5, …)
  - Word similarity

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

- Extrinsic evaluation on real tasks
  - Document classification
  - Named entity recognition

Evaluation

- Intrinsic evaluations
  - Analogy pairs (top 1, top 5, …)
  - Word similarity

- Extrinsic evaluation on real tasks
  - Document classification
  - Named entity recognition

E.g. average word embeddings in a document and apply a softmax classifier. Better score means better embeddings (in this case only semantics).

Evaluation

Named Entity Recognition

- The task: find and classify names in text, for example:

```
The European Commission [ORG] said on Thursday it
disagreed with German [MISC] advice.

Only France [LOC] and Britain [LOC] backed Fischler [PER]
's proposal .

“What we have to be extremely careful of is how other
countries are going to take Germany 's lead”, Welsh
National Farmers ' Union [ORG] ( NFU [ORG] ) chairman John
Lloyd Jones [PER] said on BBC [ORG] radio .
```

Evaluation

- Hard to work out boundaries of entity


First National Bank Donates 2 Vans To Future School Of Fort Smith

POSTED 3:43 PM, JANUARY 11, 2019, BY 5NEWS WEB STAFF

Is the first entity “First National Bank” or “National Bank”

- Hard to know if something is an entity

Is there a school called “Future School” or is it a future school?

- Hard to know class of unknown/novel entity:

To find out more about Zig Ziglar and read features by other Creators Syndicate writers and

What class is “Zig Ziglar”? (A person.)

- Entity class is ambiguous and depends on context

“Charles Schwab” is PER not ORG here! 👉

where Larry Ellison and Charles Schwab can live discreetly amongst wooded estates. And

Evaluation

- Example: Not all museums in Paris are amazing .
- Here: one true window, the one with Paris in its center and all other windows are “corrupt” in terms of not having a named entity location in their center.

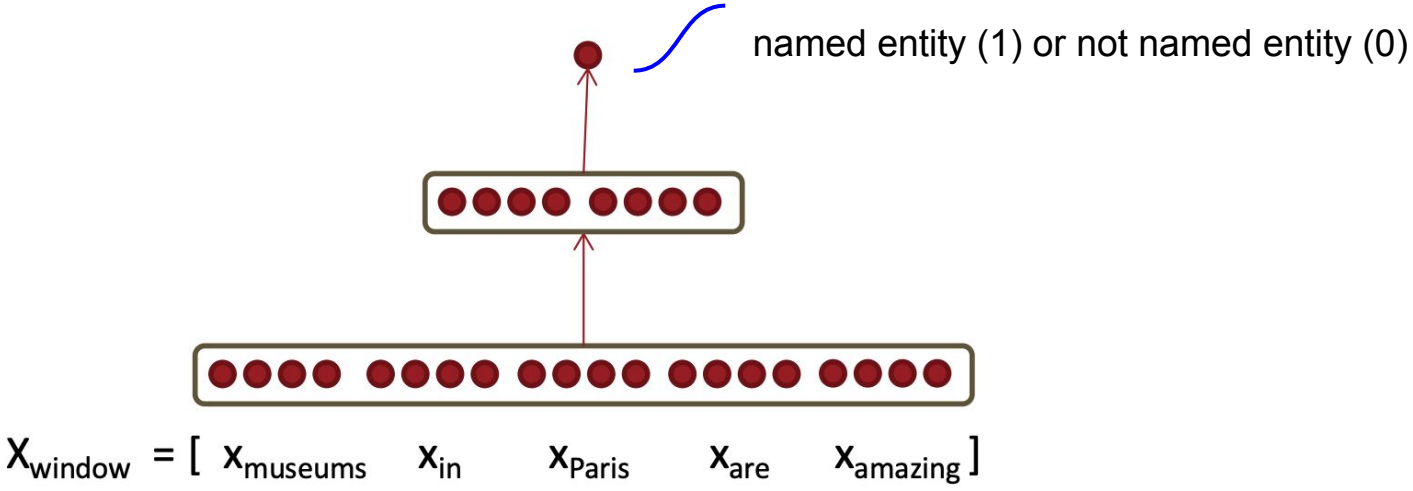
museums in Paris are amazing

- “Corrupt“ windows are easy to find and there are many: Any window whose center word isn’t specifically labeled as NER location in our corpus

Not all museums in Paris

Evaluation

- Move one word at a time across the whole labelled corpus
- Concatenate embeddings within a fixed window from a target position
- Train a network to predict whether a center word is a named entity or not
- Replace with a softmax activation for multi-class NER



Evaluation

Word	Label (IO)	Label (BIO)
Foreign	ORG	B-ORG
Ministry	ORG	I-ORG
spokesman	O	O
Shen	PER	B-PER
Guofang	PER	I-PER
told	O	O
Reuters	ORG	B-ORG
that	O	O
:	:	👆 BIO encoding

Preparing the Raw Text

- *The Question:* Before Word2Vec or GloVe can do their math, how did the computer know what the individual words were in the first place?
- Computers don't see words; they see a single, continuous string of ASCII characters: "The movie was good!"
- We must clean this string before looking up its vectors.

Tokenization

- **Definition:** The process of chopping a continuous string into standardized, manageable pieces (tokens).
- "I love AI!" -> ["I", "love", "AI", "!"]
- **Building a Vocabulary:** Once chopped, we assign a unique integer ID to every known token.
- *Example*: {"I": 1, "love": 2, "AI": 3}. This integer is what we actually feed into the computer to look up the GloVe vector.

Stop Words & Lemmatization

- **Stop Words:** Extremely common words that carry little standalone meaning (the, is, at, which).
- **Lemmatization:** Reducing words to their base dictionary form to group them together (running, ran, runs -> run).
- *The Old Paradigm:* In early Machine Learning (like Bag of Words), we ruthlessly **deleted** stop words and lemmatized everything. We did this because dictionaries were too large and computers were too slow.

Visualization